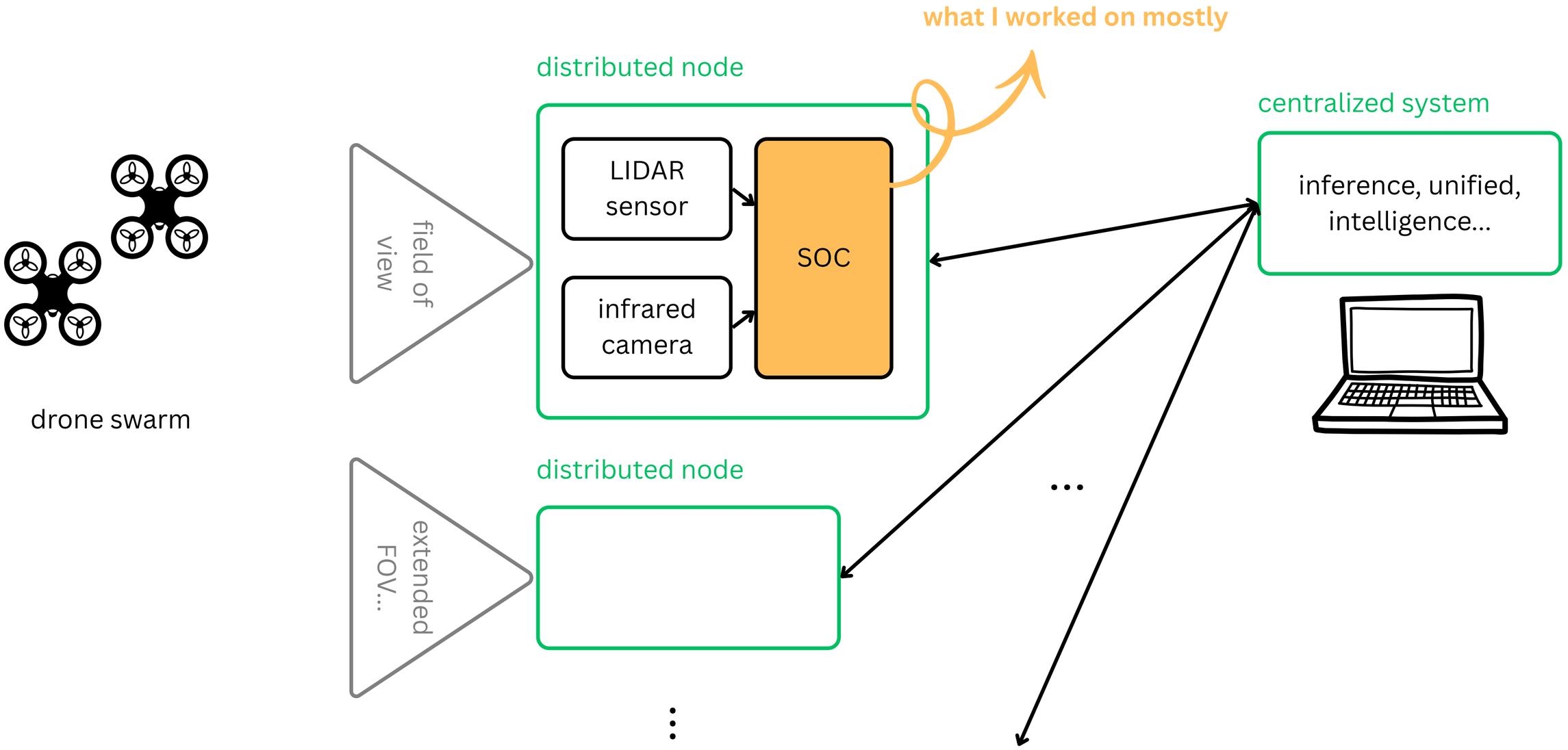
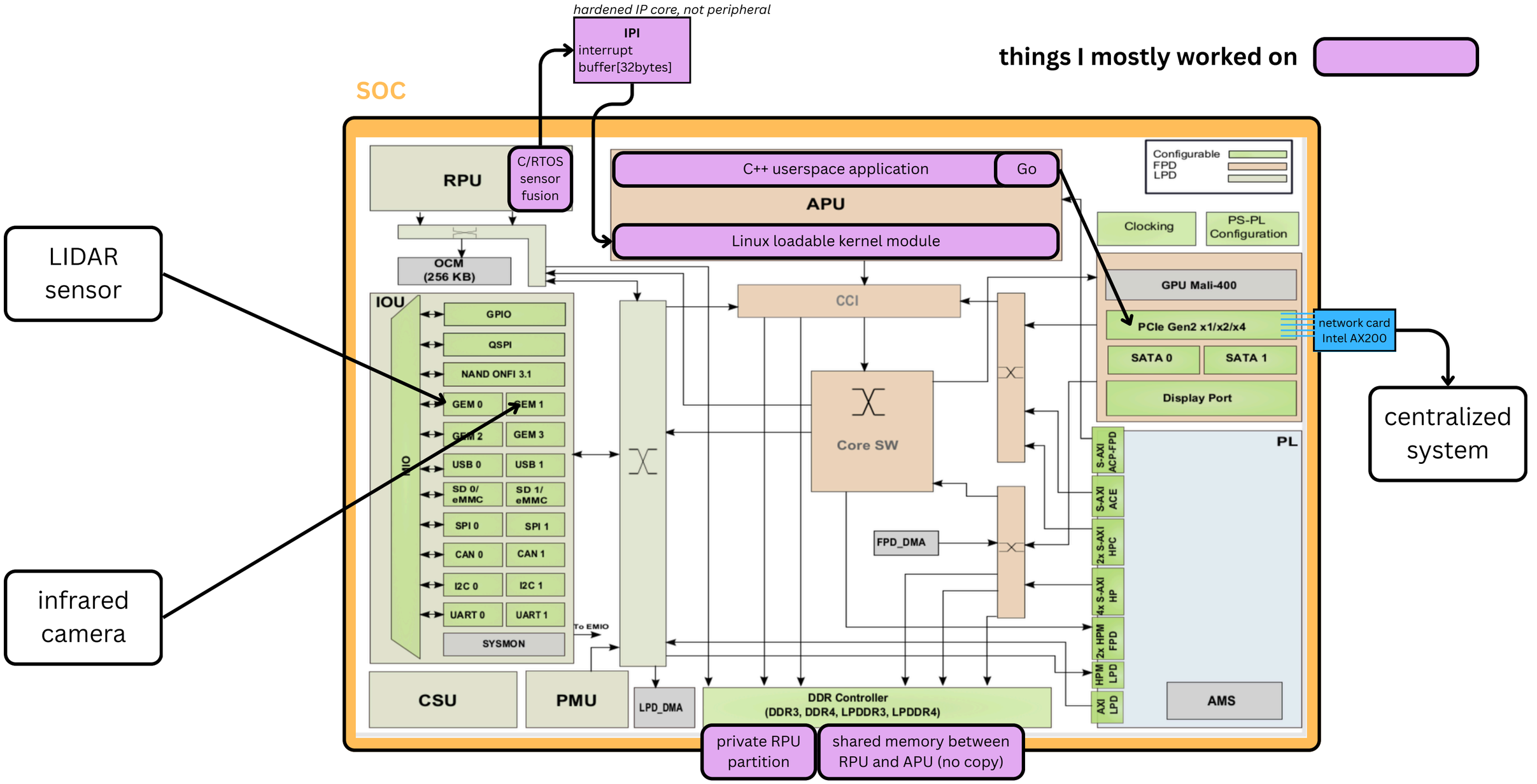
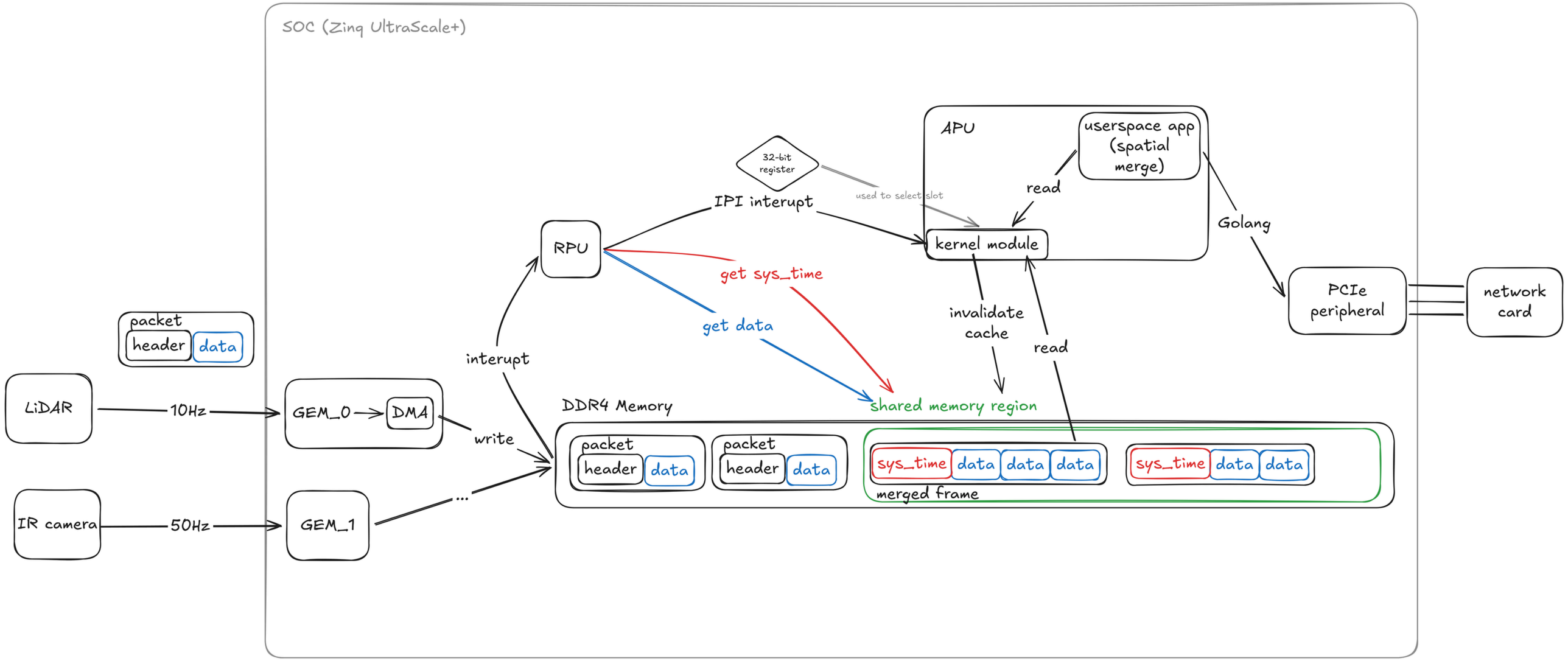
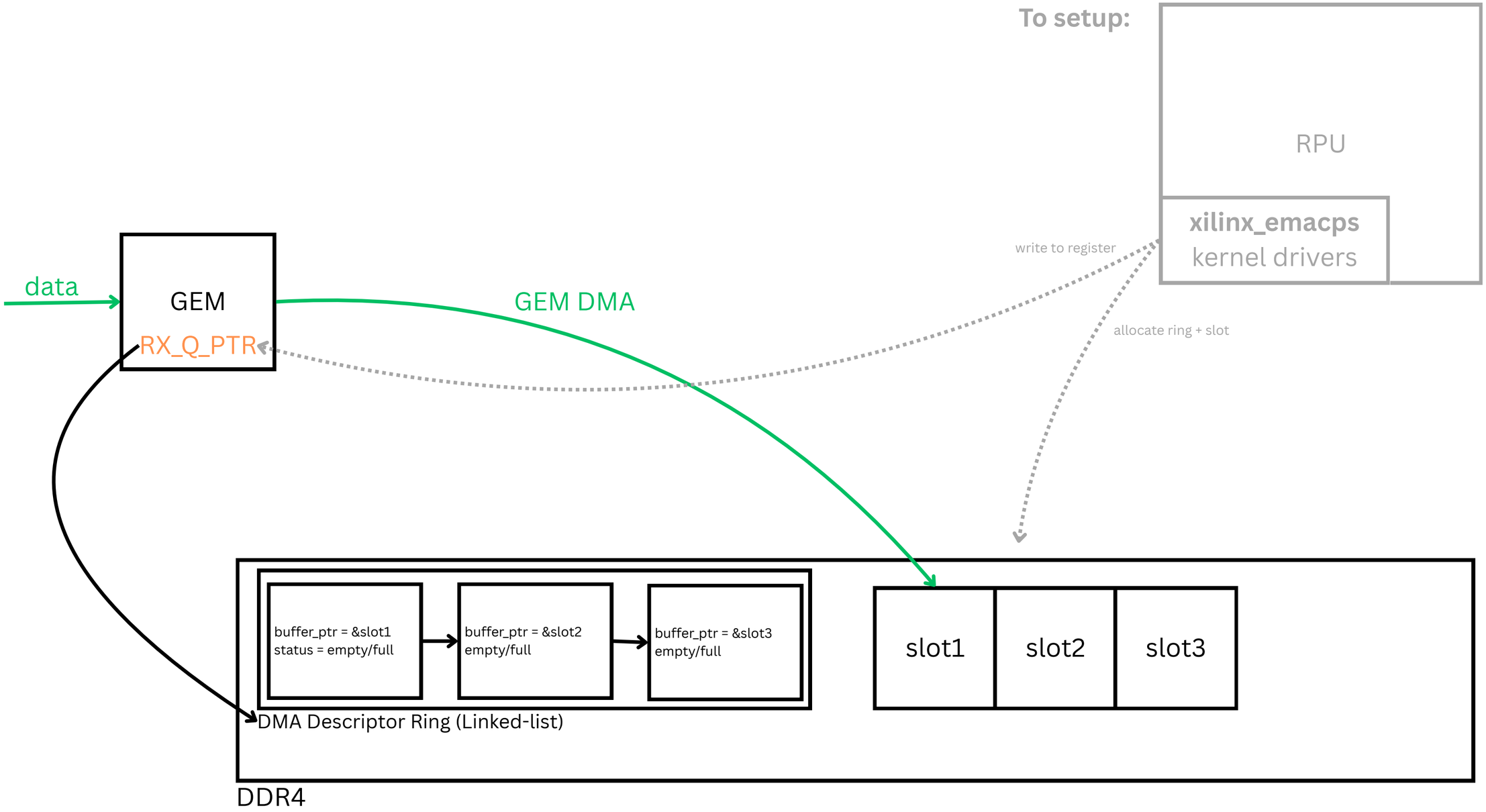
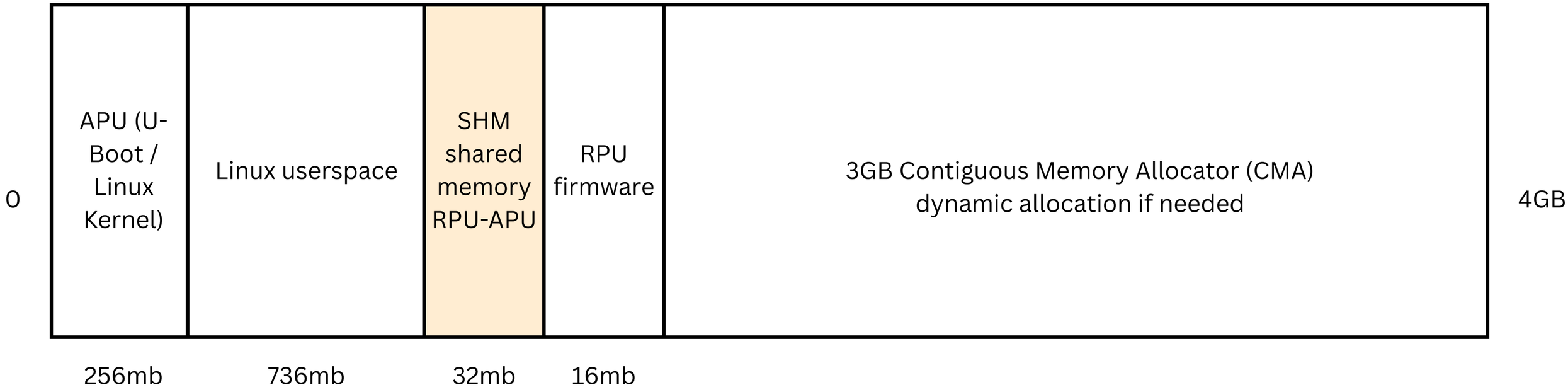

what I worked on mostly
distributed node
centralized system
LIDAR sensor
infrared camera
SOC
inference, unified, intelligence...
field of view
drone swarm
distributed node
extended FOV...


hardened IP core, not peripheral
IPI
interrupt
buffer[32bytes]
things I mostly worked on
SOC
RPU
C/RTOS
sensor
fusion
C++ userspace application
Go
APU
Linux loadable kernel module
Configurable
FPD
LPD
Clocking
PS-PL
Configuration
OCM
(256 KB)
IOU
GPIO
QSPI
NAND ONFI 3.1
GEM 0
GEM 1
GEM 2
GEM 3
USB 0
USB 1
SD 0/
eMMC
SD 1/
eMMC
SPI 0
SPI 1
CAN 0
CAN 1
I2C 0
I2C 1
UART 0
UART 1
SYSMON
EMIO
To EMIO
CCI
Core SW
FPD_DMA
GPU Mali-400
PCIe Gen2 x1/x2/x4
SATA 0
SATA 1
Display Port
PL
S-AXI
ACP-FPD
S-AXI
ACE
2x S-AXI
HPC
4x S-AXI
HP
2x HPM
FPD
HPM
LPD
AXI
LPD
CSU
PMU
LPD_DMA
DDR Controller
(DDR3, DDR4, LPDDR3, LPDDR4)
AMS
LIDAR
sensor
infrared
camera
network card
Intel AX200
centralized
system
private RPU
partition
shared memory between
RPU and APU (no copy)


SOC (Zinq UltraScale+)
APU
userspace app (spatial merge)
32-bit register
used to select slot
IPI interupt
read
Golang
RPU
kernel module
get sys_time
get data
invalidate cache
read
PCIe peripheral
network card
packet
header
data
interupt
LiDAR
10Hz
GEM_0
DMA
write
DDR4 Memory
shared memory region
packet
header
data
packet
header
data
sys_time
data
data
data
sys_time
data
data
merged frame
...
IR camera
50Hz
GEM_1


To setup:
RPU
xilinx_emacps
kernel drivers
write to register
allocate ring + slot
data
GEM
RX_Q_PTR
GEM DMA
buffer_ptr = &slot1
status = empty/full
buffer_ptr = &slot2
empty/full
buffer_ptr = &slot3
empty/full
DMA Descriptor Ring (Linked-list)
slot1
slot2
slot3
DDR4

Memory partition (DDR)



pulling: centralize system **pull** from edge device
edge device doesn't have to push continously
Benefit: rather than **push**, **pull** so that in case edge device died we know

networking on APU core 1

Exporter (Golang)

Prometheus Histogram

Bucket < 10ms: 4,000 requests
Bucket < 25ms: 12,000 requests
Bucket < 50ms: 18,500 requests
Bucket < 100ms: 19,900 requests (99% of requests finished under 100ms)
Bucket < 200ms: 20,000 requests (Total requests: 20,000)

other data, ... like prometheus.Gauge for buffer depth

http://<edge-ip>:8080/metrics

Prometheus server/database

prometheus.yml
just simply for
config how often to pull
all the IPs to pull from

HTTP GET

every 1s-5s is fine, because we have histogram

fused
done

T_start → batching... → Tstart, Coordinates{X, Y}

cal diff

kernel from here

TCP/IP ← HTTP/2 Framing ← binary

header payload

T_start, now add T_end

Got data, now send back
T_start packet

same thing but opposite

DMA move

network card
Intel AX200

network card
Intel AX200

[.proto Schema File] —(protoc compiler)→ [Generated Code: Go]

use to packs lidar/ir struct into a tiny, raw binary byte array

edge device

centralized device